

Panduan Server & Maintenance

Gudang Piala Kaltim Warehouse Management System (WMS)

Gudang Piala Kaltim WMS

Situs Operasional: wms.rionlab.space

Tanggal Diterbitkan: 13 Juli 2026

Versi: 1.0.0 (rc1)

Panduan Server & Maintenance (Server Manual)

Panduan ini berisi informasi teknis yang ditujukan bagi System Administrator, DevOps, atau Developer yang bertanggung jawab untuk memasang (setup), memelihara (maintain), mencadangkan (backup), dan memperbarui (update) sistem **Gudang Piala Kaltim WMS** di server production.

Aplikasi production dilayani pada domain: `☐ wms.rionlab.space`

Daftar Isi Panduan Server

Silakan klik tautan di bawah ini untuk mempelajari administrasi server:

1. [Arsitektur Server & Alur Ingress](#)
2. Gambaran umum teknologi server (Ubuntu 24.04 LTS + Docker Compose + Cloudflare Tunnel).
3. Penjelasan mengapa tidak membutuhkan public port forwarding.
4. [Pemasangan Baru \(Server Setup Guide\)](#)
5. Kloning repositori kode ke VPS.
6. Konfigurasi berkas `.env` dan inisialisasi server otomatis melalui skrip `setup_vps.sh`.
7. Menghubungkan Cloudflare Zero Trust Tunnel ke server lokal port 80.
8. [Operasional & Pemeliharaan Rutin](#)
9. Perintah Docker Compose untuk me-restart, mematikan, atau menghidupkan stack aplikasi.
10. Cara memeriksa log container backend dan frontend secara real-time.
11. Konfigurasi rotasi log otomatis untuk mencegah kehabisan kapasitas disk host.
12. [Pencadangan & Pemulihan \(Backup & Recovery\)](#)
13. Skrip otomatisasi backup berkala berbasis database dump PostgreSQL.
14. Penjadwalan backup otomatis melalui Cron Job di server host.
15. Prosedur verifikasi keabsahan backup (`verify_backup.sh`).
16. Prosedur pemulihan bencana (Disaster Recovery) langkah-demi-langkah.
17. [Pembaruan Sistem & Migrasi Database](#)
18. Langkah aman menarik pembaruan kode terbaru dari Git.

19. Menjalankan migrasi struktur database Postgres menggunakan Alembic di dalam container.
20. **Pemecahan Masalah (Troubleshooting)**
21. Penanganan error koneksi database, tunnel offline, atau kontainer berhenti tiba-tiba.
22. Cara mengakses dokumentasi interaktif Swagger API ([/docs](#)) di lingkungan produksi yang dilindungi oleh HTTPBasic Authentication.

****Kebijakan Keamanan:**** Server VPS tidak boleh membuka port publik apa pun kecuali port SSH yang aman. Seluruh trafik web masuk wajib dialirkan secara terenkripsi melewati terowongan Cloudflare Zero Trust Tunnel untuk mencegah ancaman pemindaian port (*port scanning*) dan serangan DDoS langsung pada alamat IP publik server.

01. Arsitektur Server & Alur Ingress

Halaman ini menjelaskan rancangan arsitektur sistem dan alur keamanan jaringan (network ingress) yang diimplementasikan pada lingkungan produksi Gudang Piala Kaltim WMS.

Spesifikasi Infrastruktur Produksi

Sistem ini didesain agar dapat berjalan dengan stabil dan aman pada spesifikasi berikut:

- **Sistem Operasi Host:** Ubuntu Server 24.04 LTS (atau versi LTS terbaru).
- **Virtualisasi:** Docker Engine v25+ & Docker Compose v2+.
- **Web Server & Reverse Proxy:** Nginx (berjalan di dalam container [frontend](#)) untuk menyajikan berkas statis SPA React, melayani routing URL Single Page Application, dan meneruskan permintaan API ke backend.
- **Database:** PostgreSQL 16 (berjalan di dalam container [db](#)).
- **Ingress Jaringan:** Cloudflare Zero Trust Tunnel ([cloudflared](#)).

Alur Trafik Masuk (Network Ingress Flow)

Sistem ini menggunakan metode **Zero Trust Tunnel** untuk menerima lalu lintas data dari domain [wms.rionlab.space](#) . Dengan metode ini, **tidak ada port publik (seperti 80 atau 443) yang dibuka pada router atau firewall VPS**. Server VPS hanya melakukan koneksi outbound (keluar) secara aman ke Cloudflare Edge.

Berikut adalah diagram visual alur trafik data dari pengguna hingga ke database:

```
sequenceDiagram
    actor User as Pengguna (Browser)
    participant CF_Edge as Cloudflare Edge (wms.rionlab.space)
    participant CF_Tunnel as Cloudflare Tunnel (cloudflared)
    participant Nginx as Nginx Container (Port 80)
    participant FastAPI as FastAPI Backend (Port 8000)
    participant Postgres as PostgreSQL (Port 5432)

    User->>CF_Edge: Request HTTPS (TLS di-terminate di Edge)
    CF_Edge->>CF_Tunnel: Kirim paket lewat terowongan aman
    CF_Tunnel->>Nginx: Forward ke http://localhost:80 (internal VPS)
    alt Request File Statis React SPA
        Nginx-->>CF_Tunnel: Kirim file index.html / assets
    else Request API (/api/v1/*)
        Nginx->>FastAPI: Proxy pass ke http://backend:8000
        FastAPI->>Postgres: Query data
        Postgres-->>FastAPI: Hasil query
        FastAPI-->>Nginx: Response JSON
        Nginx-->>CF_Tunnel: Response JSON
    end
    CF_Tunnel-->>CF_Edge: Response terenkripsi
    CF_Edge-->>User: Tampilkan di Browser Pengguna
```

Manajemen Penyimpanan Data (Persistence Strategy)

Kontainer Docker bersifat ephemeral (data di dalamnya akan hilang jika kontainer dibuat ulang). Oleh karena itu, data sensitif disimpan menggunakan **Named Volumes** yang terpasang (mounted) ke sistem file host:

1. **Volume Database (`postgres_data_prod`):**
2. Menyimpan berkas data biner PostgreSQL.
3. Terpasang di `/var/lib/postgresql/data` di dalam kontainer `db`.
4. **Volume Gambar Unggahan (`uploads_data_prod`):**
5. Menyimpan berkas foto barang yang diunggah oleh pengguna melalui aplikasi.
6. Terpasang di `/app/uploads` di dalam kontainer `backend`.

7. Ini memastikan semua foto produk tetap aman saat kontainer backend diperbarui atau di-restart.

02. Panduan Pemasangan Baru (Server Setup)

Dokumen ini menjelaskan langkah-langkah instalasi aplikasi WMS Gudang Piala Kaltim pada server Ubuntu baru, baik menggunakan skrip otomatisasi maupun secara manual, serta mengonfigurasi Cloudflare Tunnel ke domain `wms.rionlab.space`.

Prasyarat Server

- VPS Ubuntu Server 24.04 LTS terpasang di jaringan privat/publik.
- Akses SSH ke server sebagai user `root` atau user dengan hak akses `sudo`.
- Akun Cloudflare dengan domain `wms.rionlab.space` yang terdaftar.

Metode 1: Instalasi Otomatis (Sangat Direkomendasikan)

Kami menyediakan skrip otomatis `./setup_vps.sh` di root project yang akan mengonfigurasi seluruh dependensi server secara mandiri.

Langkah-Langkah:

1. Hubungkan koneksi SSH ke VPS Anda.
2. Lakukan klon repositori Git ke folder `/opt/wms`: `bash git clone https://github.com/GarionAdiwilaga/wms.git /opt/wms` `cd /opt/wms`
3. Ubah izin eksekusi skrip, lalu jalankan: `bash chmod +x setup_vps.sh` `sudo ./setup_vps.sh`
4. **Apa yang dilakukan oleh skrip ini?**
5. Memasang Docker Engine dan Docker Compose pada server host.
6. Membuat berkas `.env` produksi secara acak dan aman (termasuk generate random password PostgreSQL dan secret key JWT).
7. Meminta Anda untuk menginput kata sandi awal untuk akun administrator pertama (`INITIAL_ADMIN_PASSWORD`).
8. Memulai stack kontainer produksi (`docker-compose.prod.yml`).
9. Menjalankan migrasi database via Alembic secara otomatis.

10. Membuat pengguna awal `super_admin` dengan username `admin` dan kata sandi yang Anda tentukan tadi.
-

Metode 2: Instalasi Manual (Alternatif)

Jika Anda ingin menyiapkan lingkungan server langkah-demi-langkah secara manual:

2.1 Kloning Kode & Konfigurasi `.env`

```
git clone https://github.com/GarionAdiwilaga/wms.git /opt/wms
cd /opt/wms
cp .env.example .env
```

Edit file `.env` dan sesuaikan nilainya:

```
# Database
POSTGRES_USER=postgres
POSTGRES_PASSWORD=buat_password_db_yang_sangat_kuat
POSTGRES_DB=gudang_piala_kaltim

# Backend Settings
SECRET_KEY=buat_key_jwt_acak_panjang_minimal_32_karakter
INITIAL_ADMIN_PASSWORD=kata_sandi_akun_admin_pertama
ENVIRONMENT=production
```

2.2 Jalankan Docker Compose

Mulai build image kontainer produksi dan hidupkan layanannya:

```
docker compose -f docker-compose.prod.yml up -d --build
```

Pastikan seluruh kontainer berjalan normal:

```
docker compose -f docker-compose.prod.yml ps
```

2.3 Jalankan Migrasi Database & Buat User Admin Awal

Jalankan perintah Alembic di dalam container backend untuk membuat tabel-tabel database:

```
docker compose -f docker-compose.prod.yml exec backend alembic upgrade head
```

Jalankan skrip inisialisasi user admin awal:

```
docker compose -f docker-compose.prod.yml exec backend python scripts/create_initial_admin.py
```

(Catatan: Skrip ini akan membaca `INITIAL_ADMIN_PASSWORD` dari file `.env` Anda).

3. Konfigurasi Cloudflare Zero Trust Tunnel

Setelah aplikasi berjalan di server lokal (terikat pada `localhost:80` melalui container `frontend`), Anda harus menghubungkannya ke internet menggunakan Cloudflare Tunnel.

3.1 Install cloudflared di VPS

Ikuti petunjuk resmi Cloudflare untuk mengunduh paket instalasi. Di Ubuntu, biasanya:

```
curl -L --output cloudflared.deb https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared.deb
sudo dpkg -i cloudflared.deb
```

3.2 Login ke Cloudflare

Jalankan perintah berikut untuk mengautentikasi client:

```
cloudflared tunnel login
```

Klik link yang muncul di terminal, masuk ke dashboard Cloudflare Anda, lalu pilih domain `riionlab.space` untuk memberikan izin terowongan.

3.3 Buat Tunnel Baru

```
cloudflared tunnel create wms-tunnel
```

Perintah ini akan menghasilkan file kredensial JSON di folder `~/.cloudflared/`.

3.4 Konfigurasi Routing Domain ke Tunnel

Hubungkan domain Anda ke ID tunnel yang baru dibuat:

```
cloudflared tunnel route dns wms-tunnel wms.rionlab.space
```

3.5 Buat Berkas Konfigurasi lokal

Buat file baru di `/etc/cloudflared/config.yml` atau di `~/.cloudflared/config.yml` dengan isi:

```
tunnel: <ISI_DENGAN_ID_TUNNEL_ANDA>
credentials-file: /root/.cloudflared/<ISI_DENGAN_ID_TUNNEL_ANDA>.json

ingress:
  - hostname: wms.rionlab.space
    service: http://localhost:80
  - service: http_status:404
```

3.6 Pasang dan Jalankan cloudflared sebagai Systemd Service

Pasang terowongan sebagai layanan sistem agar otomatis aktif saat server di-reboot:

```
sudo cloudflared --config /root/.cloudflared/config.yml service install
sudo systemctl start cloudflared
sudo systemctl enable cloudflared
```

Periksa status terowongan Anda:

```
sudo systemctl status cloudflared
```

Sistem sekarang sudah online dan dapat diakses dengan aman melalui HTTPS di <https://wms.rionlab.space>.

03. Operasional & Pemeliharaan Rutin

Halaman ini berisi panduan untuk melakukan operasi harian dan pemeliharaan server secara berkala guna memastikan aplikasi berjalan tanpa gangguan (downtime).

1. Manajemen Kontainer (Docker Compose)

Semua operasi dijalankan dari direktori root aplikasi di `/opt/wms`.

- **Memeriksa Status Layanan:** `bash docker compose -f docker-compose.prod.yml ps`
 - **Me-restart Semua Layanan:** `bash docker compose -f docker-compose.prod.yml restart`
 - **Menghentikan Sementara Aplikasi:** `bash docker compose -f docker-compose.prod.yml down` (Catatan: Perintah ini menghentikan kontainer secara aman tanpa menghapus data persisten di database volume).
 - **Menjalankan Kembali Aplikasi (Detached Mode):** `bash docker compose -f docker-compose.prod.yml up -d`
-

2. Pemantauan Log Kontainer (Logging)

Kontainer Docker berjalan di latar belakang dan menuliskan log aktivitasnya. Gunakan perintah berikut untuk melacak jika ada kesalahan sistem:

- **Melihat Log Backend (API & Ledger Logik):** `bash docker compose -f docker-compose.prod.yml logs -f backend`
 - **Melihat Log Frontend (Nginx Server & Traffic Proxying):** `bash docker compose -f docker-compose.prod.yml logs -f frontend`
 - **Melihat Log Database (PostgreSQL Queries):** `bash docker compose -f docker-compose.prod.yml logs -f db`
 - **Melihat Log Gabungan (Seluruh Kontainer):** `bash docker compose -f docker-compose.prod.yml logs -f --tail 100`
-

3. Kebijakan Rotasi Log (Log Rotation Policy)

Secara default, Docker dapat menampung berkas log tanpa batas hingga kapasitas disk host penuh. Untuk menghindari hal ini, file `docker-compose.prod.yml` kami telah dikonfigurasi dengan aturan rotasi log otomatis:

```
logging:  
  driver: "json-file"  
  options:  
    max-size: "10m"  
    max-file: "3"
```

- **Artinya:** Setiap kontainer hanya akan menyimpan maksimal 3 file log dengan ukuran masing-masing tidak melebihi 10 Megabyte. Log lama akan otomatis dihapus saat batas terlampaui.

4. Pemeliharaan Cloudflare Tunnel

Karena Cloudflare Tunnel dipasang sebagai layanan sistem `systemd` pada host Ubuntu:

- **Memeriksa Keaktifan Terowongan:** `bash sudo systemctl status cloudflared`
- **Restart Layanan Tunnel (Jika koneksi internet VPS sempat terputus):** `bash sudo systemctl restart cloudflared`

5. Pemantauan Penyimpanan Disk (Disk Space Alert)

Meskipun log telah dirotasi, volume gambar unggahan dan database postgres akan terus tumbuh seiring waktu. Administrator disarankan memantau ruang kosong disk secara berkala dengan perintah:

```
df -h /
```

Jika kapasitas disk mendekati 90%, lakukan pembersihan cache docker yang tidak terpakai menggunakan:

```
docker system prune -f
```

(Perintah `prune` ini aman dijalankan dan tidak akan menghapus data biner volume database atau folder unggahan yang sedang aktif).

04. Pencadangan & Pemulihan (Backup & Recovery)

Keamanan data adalah prioritas utama. Modul ini menjelaskan kebijakan pencadangan (backup), otomatisasi penjadwalan, verifikasi kelayakan file backup, pemulihan bencana (disaster recovery), serta proses persiapan Go-Live.

1. Perintah Dasar Backup Database (PostgreSQL Dump)

Aplikasi WMS menggunakan PostgreSQL yang berjalan di dalam kontainer Docker bernama `gpk_db_prod` (sesuai `docker-compose.prod.yml`). Perintah untuk mengekspor database secara manual ke file SQL dump:

```
docker exec gpk_db_prod pg_dump -U postgres gudang_piala_kaltim > /var/backups/wms/wms_backup_$(date +%F)
```

2. Penjadwalan Backup Otomatis (Cron Job)

Disarankan untuk melakukan backup harian otomatis pada jam sepi aktivitas (misalnya pukul 01:00 dini hari).

Cara Mengonfigurasi Cron Job di Ubuntu VPS Host:

1. Masuk ke terminal host dan jalankan editor cron: `bash sudo crontab -e`
 2. Tambahkan baris konfigurasi berikut di bagian paling bawah file: `cron 0 1 * * * docker exec gpk_db_prod pg_dump -U postgres gudang_piala_kaltim > /var/backups/wms/wms_backup_$(date +%F).sql 2>&1`
 3. Simpan dan keluar dari editor. Layanan cron sekarang akan membuat file backup baru setiap hari pada pukul 01:00.
 4. **Kebijakan Retensi:** Pastikan Anda memiliki script untuk membersihkan file backup yang berumur lebih dari 30 hari guna menghemat ruang disk host (misalnya menggunakan perintah `find /var/backups/wms/ -mtime +30 -type f -delete`).
-

3. Uji Coba Verifikasi Backup (Rehearsal)

Backup data tidak ada gunanya jika file dump yang dihasilkan rusak dan tidak bisa di-restore. Kami menyediakan skrip otomatis untuk menyimulasikan siklus pemulihan data pada database sementara (`wms_verify_temp`).

Cara Menjalankan Verifikasi:

Jalankan perintah berikut langsung dari shell VPS Anda:

```
/opt/wms/backend/scripts/verify_backup.sh
```

Apa yang dilakukan skrip ini? 1. Membuat backup live database saat ini. 2. Membuat database sementara bernama `wms_verify_temp` di PostgreSQL. 3. Mencoba mengimpor berkas backup tersebut ke database sementara. 4. Menjalankan query pengecekan integritas tabel untuk memastikan struktur data utuh. 5. Menghapus database sementara tersebut. 6. Menampilkan output hasil verifikasi. Pastikan skrip mengembalikan pesan: `BACKUP VERIFICATION SUCCESSFUL!`

4. Prosedur Pemulihan Bencana (Disaster Recovery)

Jika database utama rusak, terhapus, atau mengalami korupsi data, ikuti langkah-langkah pemulihan berikut dengan sangat teliti:

1. **Hentikan Layanan Backend** terlebih dahulu agar tidak ada koneksi API baru yang masuk selama pemulihan: `bash docker compose -f /opt/wms/docker-compose.prod.yml stop backend`
2. **Hapus database lama dan buat database kosong baru** di dalam kontainer DB:
`bash docker exec -i gpk_db_prod psql -U postgres -c "DROP DATABASE gudang_piala_kaltim;"`
`docker exec -i gpk_db_prod psql -U postgres -c "CREATE DATABASE gudang_piala_kaltim;"`
3. **Impor data dari berkas SQL dump pilihan Anda** (misal backup dari tanggal kemarin): `bash cat /var/backups/wms/wms_backup_YYYY-MM-DD.sql | docker exec -i gpk_db_prod psql -U postgres -d gudang_piala_kaltim`
4. **Aktifkan kembali layanan backend:** `bash docker compose -f /opt/wms/docker-compose.prod.yml start backend`
5. Buka aplikasi di browser untuk memverifikasi bahwa data telah pulih ke titik waktu backup tersebut.

5. Persiapan Go-Live (Reset Transaksi UAT/Pilot)

Jika Anda telah selesai melakukan pengujian sistem (UAT / User Acceptance Testing) atau masa uji coba Pilot run, dan ingin menghapus seluruh data transaksi palsu sebelum aplikasi digunakan secara resmi (Go-Live) **tanpa menghapus data master (Items, Categories, Suppliers, Users, Branches)**:

Jalankan perintah script khusus di kontainer backend:

```
docker compose -f /opt/wms/docker-compose.prod.yml exec backend python scripts/prepare_go_live.py
```

• Perilaku Script:

- Menghapus seluruh riwayat transaksi di ledger (`inventory_transactions`).
- Mengosongkan keranjang outbound dan draft opname/transfer.
- Mengosongkan tabel `transfers` , `transfer_lines` , dan `stock_opnames` .
- Mereset nilai cache stok barang di semua cabang (`branch_stocks`) menjadi 0.
- Mereset seluruh sequence ID tabel agar ID transaksi kembali dimulai dari angka 1.
- **PENTING:** Script ini dilengkapi dengan konfirmasi interaktif keamanan untuk mencegah eksekusi yang tidak disengaja. Ikuti instruksi di layar untuk melanjutkan.

05. Pembaruan Sistem & Migrasi Database

Dokumen ini menjelaskan prosedur standar untuk memperbarui kode aplikasi WMS di server produksi dan mengaplikasikan perubahan struktur tabel database (migrasi schema) secara aman tanpa merusak data yang ada.

Prosedur Standard Pembaruan Aplikasi (Standard Update Flow)

****Pra-Pembaruan (Pre-update Backup):**** Sebelum melakukan penarikan kode terbaru dari Git atau menjalankan migrasi database, ****lakukan backup database secara manual terlebih dahulu**** menggunakan prosedur di [Panduan Backup](./04_backup_recovery.md#1-perintah-dasar-backup-database-postgresql-dump). Hal ini memastikan Anda memiliki titik pemulihan jika pembaruan baru mengalami kegagalan fatal.

Ikuti urutan langkah berikut di terminal VPS:

Langkah 1: Ambil Kode Terbaru dari Repositori Git

Masuk ke direktori aplikasi dan lakukan penarikan (pull) kode:

```
cd /opt/wms  
git pull origin main
```

(Catatan: Jika ada revisi tag rilis tertentu, Anda dapat berpindah ke tag tersebut dengan `git checkout tags/vX.Y.Z`).

Langkah 2: Bangun Ulang Kontainer Docker (Rebuild Stack)

Jalankan perintah build untuk memperbarui image kontainer (Nginx frontend & FastAPI backend) dengan kode yang baru ditarik:

```
docker compose -f docker-compose.prod.yml up -d --build
```

- Perintah ini akan membangun kembali image yang mendeteksi perubahan berkas statis React maupun script Python backend.
- Layanan kontainer akan digantikan (replace) secara otomatis dengan downtime yang sangat minimal (hitungan detik).

2. Migrasi Database (Database Migration via Alembic)

Jika pembaruan kode baru menyertakan perubahan struktur tabel database (misal penambahan kolom baru atau tabel baru), Anda harus menerapkan migrasi database menggunakan Alembic.

Alembic berjalan di dalam kontainer `backend`.

Langkah 2.1: Periksa Riwayat Migrasi (Opsional)

Untuk melihat riwayat migrasi yang sudah pernah diterapkan pada database:

```
docker compose -f docker-compose.prod.yml exec backend alembic history
```

Langkah 2.2: Jalankan Peningkatan Struktur Database (Upgrade Head)

Gunakan perintah berikut untuk menaikkan versi schema database ke versi paling baru yang terdaftar di berkas migrasi:

```
docker compose -f docker-compose.prod.yml exec backend alembic upgrade head
```

Jika migrasi berhasil, sistem akan mencetak log peningkatan dari versi sebelumnya menuju versi target dengan tulisan `Running upgrade ...`.

3. Langkah Pasca-Pembaruan (Post-update Validation)

Setelah pembaruan selesai dilakukan: 1. Bersihkan cache browser Anda (terutama jika ada pembaruan visual yang tidak muncul secara instan karena cache React SPA). 2. Akses <https://wms.rionlab.space> dan pastikan halaman Login muncul dengan benar. 3. Masuk ke halaman **Dashboard** dan periksa apakah data stok barang dan log aktivitas tetap utuh dan akurat. 4. Periksa log kontainer backend untuk memastikan tidak ada error startup aplikasi: `bash docker compose -f docker-compose.prod.yml logs -f backend`

06. Pemecahan Masalah (Troubleshooting)

Halaman ini mendokumentasikan kendala umum yang mungkin terjadi selama pengoperasian server Gudang Piala Kaltim WMS beserta langkah penyelesaiannya.

1. Aplikasi Menampilkan Error "502 Bad Gateway" di Browser

Pesan error ini berasal dari Cloudflare Tunnel atau Nginx yang tidak dapat menghubungi layanan di belakangnya (biasanya kontainer backend/frontend mati).

Langkah Deteksi:

1. SSH ke VPS Anda.
2. Periksa status kontainer: `bash docker compose -f /opt/wms/docker-compose.prod.yml ps`
3. **Skenario A: Kontainer `gpk_backend_prod` atau `gpk_frontend_prod` berstatus `Exit` :**
4. Lihat log untuk mendeteksi penyebab matinya kontainer: `bash docker compose -f /opt/wms/docker-compose.prod.yml logs backend`

5. Hidupkan kembali layanan: `bash docker compose -f /opt/wms/docker-compose.prod.yml up -d`

6. Skenario B: Kontainer DB mati / restart terus menerus:

7. Hal ini biasanya dikarenakan error inisialisasi volume data database atau kerusakan file biner postgres. Periksa log database: `bash docker compose -f /opt/wms/docker-compose.prod.yml logs db`

2. Koneksi Database Error (FastAPI gagal terhubung ke Postgres)

Di log backend muncul pesan `psycopg2.OperationalError: connection to server at "db" ... failed`.

Langkah Penyelesaian:

1. Pastikan kontainer database `gpk_db_prod` dalam keadaan sehat (`Up`).
2. Periksa kembali isi berkas konfigurasi `/opt/wms/.env` . Pastikan password database `POSTGRES_PASSWORD` cocok dengan nilai konfigurasi di backend.
3. Jika password baru saja diubah di file `.env` , kontainer database harus dimatikan terlebih dahulu secara penuh agar perubahan dapat di-apply ke internal volume: `bash docker compose -f /opt/wms/docker-compose.prod.yml down docker compose -f /opt/wms/docker-compose.prod.yml up -d`

3. Situs Tidak Dapat Diakses Secara Keseluruhan (Cloudflare Tunnel Offline)

Jika domain `wms.rionlab.space` menampilkan pesan error "Cloudflare Tunnel Error 1033", artinya agen `cloudflared` di VPS Anda mati atau gagal melakukan koneksi ke Cloudflare Edge.

Langkah Penyelesaian:

1. Periksa status layanan `cloudflared` di host VPS: `bash sudo systemctl status cloudflared`
2. Jika berstatus `inactive` atau `failed` , nyalakan kembali layanan: `bash sudo systemctl restart cloudflared`
3. Jika masih offline, periksa log koneksi tunnel: `bash sudo journalctl -u cloudflared -f -n 50`

4. Kemungkinan penyebab: VPS kehilangan koneksi internet, berkas konfigurasi tunnel di `/root/.cloudflared/` terhapus, atau masa berlaku token tunnel dari dashboard Cloudflare Zero Trust telah kedaluwarsa.
-

4. Akses Dokumentasi API (Swagger /docs) di Produksi

Secara default di FastAPI, dokumentasi interaktif Swagger (`/docs` , `/redoc` , `/openapi.json`) sering kali dimatikan pada mode produksi demi alasan keamanan. Namun, pada sistem ini, Swagger tetap dinyalakan tetapi **dilindungi secara ketat di belakang HTTPBasic Authentication**.

- **Alamat Swagger:** `https://wms.rionlab.space/docs`
- **Kredensial Akses:**
 - Saat halaman dibuka, browser akan memunculkan dialog pop-up berisi kolom Username dan Password.
 - **Username:** `admin` (atau username lain yang memiliki peran `super_admin`).
 - **Password:** Sandi yang sesuai dengan akun `super_admin` tersebut.
- **Aturan Teknis:**
 - Di lingkungan pengembangan (`ENVIRONMENT=development`), Swagger dapat diakses secara bebas tanpa dialog autentikasi dasar (HTTPBasic Auth).
 - Di lingkungan produksi (`ENVIRONMENT=production`), verifikasi akun `super_admin` wajib lolos sebelum dokumen API dapat dimuat ke layar.